

Análisis y Diseño de Aplicaciones II

Trabajo Final de Unidad III

Septiembre 2026

Constanza Dibueno

Luis Gaudiano

Guillermo Acquistapace

Micaela Crocco

1 Introducción y alcance

InvestNow es la plataforma de inversión que definimos al comienzo del curso. Para esta unidad tomamos una parte acotada de ese dominio y la usamos para mostrar una solución formada por varios componentes que se comunican entre sí. El alcance se concentra en la consulta de activos, la administración de saldos y el registro de órdenes de compra.

Dividimos la solución en tres componentes basándonos en las responsabilidades principales que tiene el dominio. De esta forma, Catálogo se ocupa de los activos disponibles, Wallet administra el dinero de los usuarios y Órdenes coordina el proceso de compra. Cada componente expone una API REST y puede ejecutarse en su propio contenedor.

En el informe presentamos el modelo de componentes, la partición que elegimos y el proceso que seguimos para llegar a esa división. También explicamos por qué usamos contenedores y transacciones ACID, y qué impacto tendría utilizar máquinas virtuales o un modelo BASE en esta solución.

2 Modelo de componentes

2.1 Vista general

El modelo está formado por los servicios de Catálogo, Wallet y Órdenes. Catálogo y Wallet ofrecen información y operaciones específicas de su área, mientras que Órdenes consume esas interfaces para completar una compra. Esta relación permite que cada componente mantenga una responsabilidad concreta y que la comunicación entre ellos quede definida mediante contratos REST.

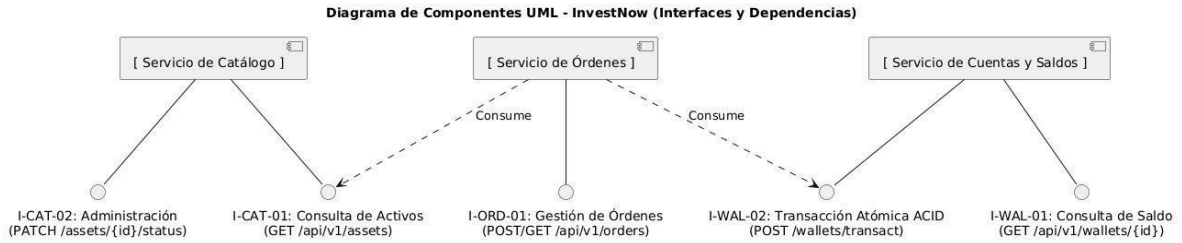


Figura 1: Modelo de componentes de InvestNow

2.2 Componentes

Servicio de Catálogo

El Servicio de Catálogo administra los activos que se pueden operar en la plataforma. Expone una interfaz para consultar la lista de activos y otra para cambiar su estado entre activo e inactivo. No necesita consumir interfaces de otros componentes para cumplir esta responsabilidad. En la demostración, el catálogo utiliza datos en memoria y su consulta puede atenderse desde varias réplicas.

Servicio de Cuentas y Saldos

El Servicio de Cuentas y Saldos, al que llamamos Wallet en la implementación, administra el saldo de cada usuario. Permite consultar un saldo y aplicar débitos o créditos. Los datos se guardan en PostgreSQL y las modificaciones se realizan dentro de una transacción. El servicio no guarda el saldo dentro del proceso, por lo que puede reiniciarse sin perder la información.

Servicio de Órdenes

El Servicio de Órdenes recibe las solicitudes de compra. Primero consulta al Servicio de Catálogo para comprobar que el activo exista y esté habilitado. Después solicita al Servicio de Wallet que debite el importe correspondiente. Por este motivo, Órdenes consume interfaces de los otros dos componentes y actúa como coordinador del caso de uso.

2.3 Interfaces y dependencias

Esta tabla muestra de forma resumida las interfaces que expone cada componente, quién las consume y qué función van a tener dentro de nuestra solución.

Interfaz	Componente que la expone	Consumidor	Función
I-CAT-01	Catálogo	Órdenes y clientes	Consultar activos
I-CAT-02	Catálogo	Administrador	Modificar el estado de un activo
I-WAL-01	Wallet	Clientes	Consultar un saldo
I-WAL-02	Wallet	Órdenes	Aplicar un débito o crédito
I-ORD-01	Órdenes	Clientes	Registrar y consultar órdenes

La dependencia principal parte del Servicio de Órdenes. Para aceptar una compra necesita consultar Catálogo y luego solicitar el débito a Wallet. Catálogo y Wallet no dependen de Órdenes ni se comunican directamente entre ellos. Esto evita dependencias circulares y deja claro qué componente inicia cada interacción.

3 Partición de primer nivel

Para la partición de primer nivel usamos una partición por dominio. Esto significa que agrupamos el sistema de acuerdo con las responsabilidades del negocio y con la información que maneja cada parte. Así surgieron las áreas de Catálogo, Cuentas y Saldos, y Órdenes.

Elegimos esta partición porque las tres áreas tienen necesidades diferentes. Catálogo concentra operaciones de consulta y puede necesitar varias réplicas para atender más solicitudes. Wallet modifica datos financieros y necesita mantener el saldo consistente. Órdenes aplica las reglas del proceso de compra y coordina las llamadas a los otros servicios. Separarlas permite modificar o escalar una parte sin desplegar toda la aplicación como una sola unidad.

Una alternativa hubiera sido aplicar una partición técnica y separar, por ejemplo, todos los controladores, toda la lógica de negocio y todo el acceso a datos. Esa división organiza el código por capas, pero hace que una funcionalidad atraviese varias partes de la solución. Para esta demostración nos resultó más útil mantener juntas las operaciones relacionadas con cada área del negocio y comunicar esas áreas mediante interfaces definidas.

4 Proceso para encontrar los componentes

El punto de partida fueron las historias de usuario definidas para InvestNow. Dentro del alcance de esta unidad nos concentramos en consultar activos, ver el saldo y registrar una orden de compra. A partir de esos casos de uso identificamos qué responsabilidades aparecían y qué datos necesitaba manejar cada una.

La consulta y administración de activos forman un grupo porque trabajan sobre el mismo catálogo. La consulta y modificación del saldo forman otro grupo, con reglas de consistencia propias. El registro de órdenes quedó separado porque necesita coordinar las dos áreas anteriores, pero no debería administrar directamente ni los activos ni los saldos.

Una vez definidos esos grupos, revisamos qué información debía cruzar sus límites. De ahí surgieron las interfaces REST: Órdenes necesita consultar los activos y solicitar una transacción sobre el saldo. El resultado fue un modelo de tres componentes con responsabilidades diferenciadas y dos dependencias principales.

5 Contenedores y máquinas virtuales

5.1 Uso de contenedores

Usamos contenedores porque sirven para correr cada servicio con todas sus dependencias y para que tengamos el mismo entorno mediante Docker Compose. La solución cuenta con un contenedor para cada servicio y otro para PostgreSQL. Docker Compose define cómo se construyen, qué variables de entorno usan y cómo se comunican dentro de la red del proyecto.

Esta opción también facilita la demostración. El entorno completo puede levantarse con un solo comando y no depende de que cada máquina tenga instaladas las mismas versiones de Python, PostgreSQL y sus bibliotecas. Además, es posible crear nuevas instancias de Catálogo para mostrar escalabilidad horizontal sin tener que preparar un sistema operativo nuevo para cada réplica.

5.2 Impacto de utilizar máquinas virtuales

Si usaramos máquinas virtuales, cada instancia tendría su propio sistema operativo además de la aplicación. Esto aumentaría el consumo de memoria y almacenamiento, y también obligaría a configurar el sistema operativo, el entorno de ejecución y la red de cada máquina. Para repetir la demostración necesitaríamos automatizar ese aprovisionamiento o documentar varios pasos adicionales.

Las máquinas virtuales aportarían un aislamiento mayor porque no compartirían el sistema operativo anfitrión. Igualmente ese nivel de aislamiento no es necesario para el alcance de esta solución. También haría más costoso crear varias instancias de Catálogo, por lo que la demostración de escalabilidad sería menos directa que con contenedores.

6 ACID y BASE

6.1 Uso de transacciones ACID

Nosotros elegimos usar transacciones ACID para las operaciones de saldo del Servicio de Wallet. Estas propiedades permiten que una operación se aplique completa o no se aplique, mantenga las reglas de los datos, quede aislada de otras operaciones concurrentes y permanezca guardada después de confirmarse. Cuando llega un débito o un crédito, el servicio inicia una transacción en PostgreSQL y bloquea la fila del usuario mientras calcula y guarda el nuevo saldo. Si el usuario no existe, el saldo resulta negativo o se produce otro error, la operación se revierte y el valor anterior se mantiene.

El bloqueo evita que dos solicitudes concurrentes lean el mismo saldo y se sobrescriban entre sí. Cada transacción termina antes de que la siguiente pueda modificar esa cuenta. Esto es importante porque un resultado incorrecto afectaría directamente el dinero disponible del usuario.

En esta demostración, el alcance de ACID está dentro de Wallet. La consulta del activo, el débito y el registro de la orden ocurren en servicios distintos y no forman una única transacción distribuida. Resolver ese flujo completo requeriría mecanismos adicionales y queda fuera del alcance definido para esta unidad.

6.2 Impacto de utilizar BASE

Con un modelo BASE podríamos priorizar la disponibilidad y aceptar que las copias del saldo no se actualicen al mismo tiempo. Durante ese intervalo, una solicitud podría leer un valor anterior y aceptar una compra que ya no tiene fondos suficientes. Después sería necesario detectar la diferencia y aplicar una corrección o una operación compensatoria.

Ese modelo puede ser apropiado para información que admite una demora en la sincronización, pero introduce una complejidad innecesaria para el saldo disponible. En InvestNow preferimos que una operación pueda esperar brevemente por el bloqueo antes que confirmar una modificación basada en datos desactualizados. Por ese motivo usamos ACID en la parte financiera de la demostración.

7 Conclusión

Como usamos una partición por dominio pudimos separar Catálogo, Wallet y Órdenes de acuerdo con las responsabilidades que cumplen dentro de InvestNow. Los contenedores sirven para que el entorno sea fácil de reproducir y permiten agregar réplicas de Catálogo, mientras que PostgreSQL y las transacciones ACID mantienen consistentes las operaciones de saldo. El alcance que elegimos no implementa toda la plataforma, pero es una buena dimensión para mostrar los componentes, sus interfaces y las decisiones arquitectónicas que tomamos.